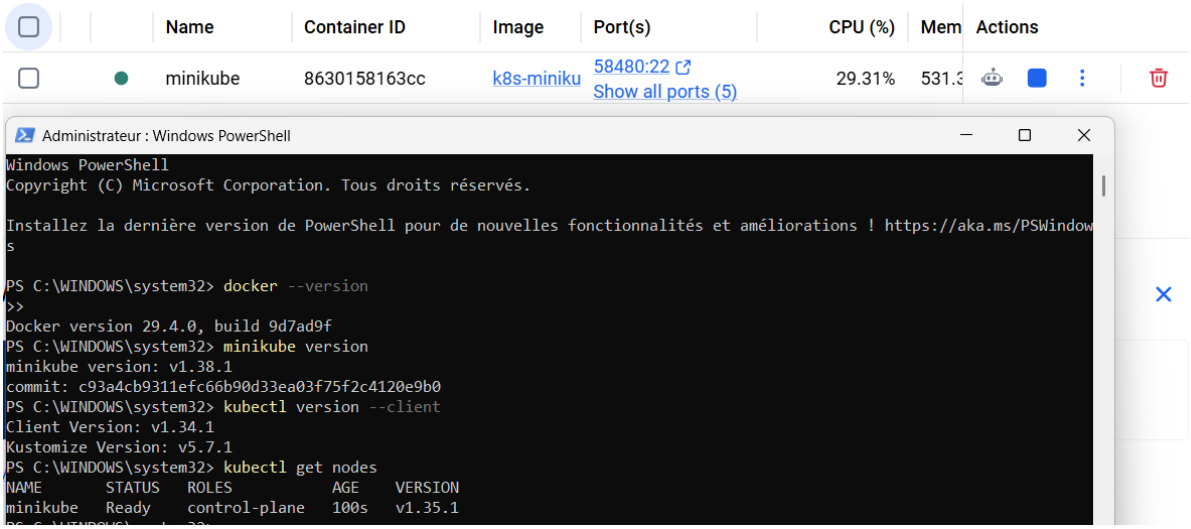
TP Minikube :

Démarrage du cluster Kubernetes local



Création du déploiement

```
PS C:\WINDOWS\system32> kubectl create deployment web --image=nginx
deployment.apps/web created
```

Afficher les pods :

```
PS C:\WINDOWS\system32> kubectl get deployments
NAME    READY    UP-TO-DATE    AVAILABLE    AGE
web     1/1      1             1            48s
PS C:\WINDOWS\system32> kubectl get pods
NAME                    READY    STATUS     RESTARTS    AGE
web-68d995574f-kc4n2    1/1      Running    0           66s
```

Création de 3 instances :

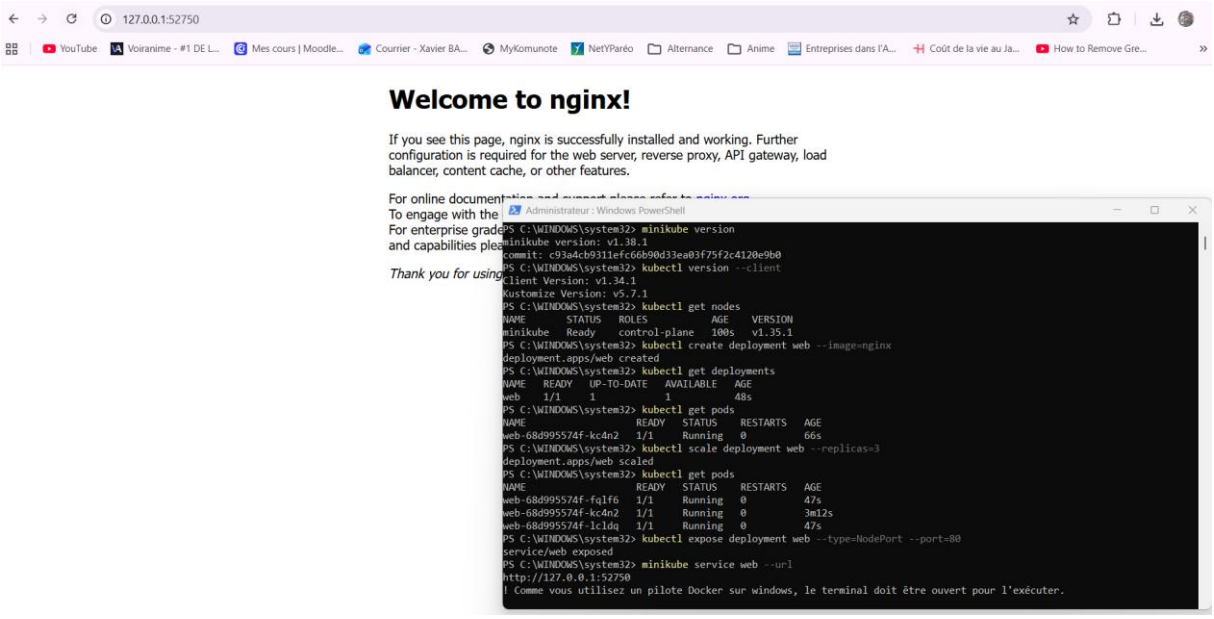
```
PS C:\WINDOWS\system32> kubectl scale deployment web --replicas=3
deployment.apps/web scaled
```

Afficher les pods :

```
PS C:\WINDOWS\system32> kubectl get pods
NAME                    READY    STATUS     RESTARTS    AGE
web-68d995574f-fqlf6    1/1      Running    0           47s
web-68d995574f-kc4n2    1/1      Running    0           3m12s
web-68d995574f-lcldq    1/1      Running    0           47s
```

On expose le service pour y accéder :

Récupérer l'URL :



Supprimer un pod :

```
PS C:\WINDOWS\system32> kubectl get pods
NAME                    READY   STATUS    RESTARTS   AGE
web-68d995574f-fqlf6    1/1     Running   0          4m10s
web-68d995574f-kc4n2    1/1     Running   0          6m35s
web-68d995574f-lcldq    1/1     Running   0          4m10s
PS C:\WINDOWS\system32> kubectl delete pod web-68d995574f-fqlf6
pod "web-68d995574f-fqlf6" deleted from default namespace
PS C:\WINDOWS\system32> kubectl get pods
NAME                    READY   STATUS    RESTARTS   AGE
web-68d995574f-fns7x    1/1     Running   0          6s
web-68d995574f-kc4n2    1/1     Running   0          7m2s
web-68d995574f-lcldq    1/1     Running   0          4m37s
```

8. Questions

1. La différence entre un système centralisé et distribué

Un **système** est un ensemble de composants informatiques travaillant pour un même but. Dans une architecture **centralisée**, toutes les données et décisions passent par un unique **nœud** (une unité de calcul ou serveur). Si ce nœud subit une panne, le service entier s'arrête : c'est un **point de défaillance unique**. À l'inverse, un système **distribué** répartit les tâches sur plusieurs nœuds indépendants reliés par un réseau. Cette structure élimine la dépendance à une seule machine ; si l'une tombe, les autres compensent, ce qui assure une meilleure continuité du service.

2. L'utilisation de plusieurs instances d'une application

Une **instance** est un exemplaire spécifique d'un logiciel en cours d'exécution sur un serveur. Utiliser plusieurs instances simultanément permet de gérer la **charge** en la répartissant. C'est ce qu'on appelle la **scalabilité horizontale** : au lieu d'avoir un seul serveur surpuissant, on multiplie les petites unités de travail. Cela protège aussi l'application : si une instance s'arrête brusquement à cause d'une erreur logicielle, les autres instances identiques continuent de répondre aux utilisateurs sans interruption.

3. La gestion d'une panne de Pod

Dans l'écosystème Kubernetes, un **Pod** est la plus petite unité d'exécution, une sorte de capsule contenant l'application. Kubernetes fonctionne selon un modèle d'**état désiré** : l'administrateur définit le nombre de Pods qui doivent fonctionner en permanence. Si un Pod tombe en panne, le système détecte immédiatement une différence avec l'état réel. Il déclenche alors un mécanisme d'**auto-guérison** en supprimant le Pod défectueux et en recréant un nouveau Pod sur un serveur sain pour rétablir la configuration demandée.

4. La tolérance aux fautes

La **tolérance aux fautes** désigne la capacité d'un système informatique à rester opérationnel malgré la survenue d'une **faute**. Elle repose sur la **redondance**, c'est-à-dire le fait de posséder des composants en surplus déjà actifs. Contrairement à une simple réparation après panne, un système tolérant aux fautes masque l'incident à l'utilisateur : le passage du composant défectueux au composant de secours est instantané et transparent.

5. Kubernetes et la haute disponibilité

La **haute disponibilité** est l'objectif technique visant à garantir qu'un service est accessible quasiment 100 % du temps. Kubernetes ne garantit pas la haute disponibilité de manière automatique par sa seule installation ; il est un **orchestrateur**, un outil de gestion. Pour obtenir une réelle disponibilité, l'infrastructure doit être configurée avec une **redondance géographique** et sur plusieurs **nœuds maîtres** . Si Kubernetes est installé sur une machine unique qui tombe en panne, l'ensemble du système devient indisponible.

6. Le rôle du load balancing

Le **load balancing**, ou équilibrage de charge, est assuré par un composant qui agit comme un aiguilleur entre les utilisateurs et les serveurs. Lorsqu'une **requête** arrive, l'équilibreur choisit l'instance la plus apte à y répondre selon des algorithmes précis. Son rôle est double : il optimise l'utilisation des ressources en évitant la surcharge d'un seul serveur, et il effectue des **health checks** (vérifications de santé). S'il détecte qu'une instance est malade, il cesse de lui envoyer du trafic, garantissant ainsi que l'utilisateur n'aboutisse jamais sur une page d'erreur.

9. Bonus :

Augmenter le nombre de réplicas :

```
PS C:\WINDOWS\system32> kubectl scale deployment web --replicas=5
deployment.apps/web scaled
PS C:\WINDOWS\system32> kubectl get pods
NAME                    READY   STATUS              RESTARTS   AGE
web-68d995574f-957cv    0/1     ContainerCreating   0          4s
web-68d995574f-jw4ld    0/1     ContainerCreating   0          4s
web-68d995574f-kc4n2    1/1     Running             0          27m
web-68d995574f-lcldq    1/1     Running             0          24m
web-68d995574f-zrvfx    1/1     Running             0          18m
PS C:\WINDOWS\system32> kubectl get pods
NAME                    READY   STATUS    RESTARTS   AGE
web-68d995574f-957cv    1/1     Running   0          9s
web-68d995574f-jw4ld    1/1     Running   0          9s
web-68d995574f-kc4n2    1/1     Running   0          27m
web-68d995574f-lcldq    1/1     Running   0          24m
web-68d995574f-zrvfx    1/1     Running   0          18m
```

L'augmentation du nombre de **réplicas** qui sont des copies identiques d'une application fonctionnant simultanément améliore la **fiabilité** du système, c'est-à-dire sa capacité à fonctionner sans interruption. Toutefois, cette corrélation n'est pas infinie et dépend de plusieurs facteurs structurels. Augmenter les réplicas renforce la résistance aux pannes aléatoires et matérielles, mais la fiabilité globale dépend surtout de la stratégie de distribution de ces copies et de la qualité du code partagé.

10. Nettoyage

Supprimer le service :

```
PS C:\WINDOWS\system32> kubectl delete service web
service "web" deleted from default namespace
PS C:\WINDOWS\system32> kubectl delete deployment web
>>
deployment.apps "web" deleted from default namespace
PS C:\WINDOWS\system32> minikube stop
>>
* Arrêt du nœud  "minikube" ...
* Mise hors tension du profil "minikube" via SSH…
```

Notion technique :

Cluster : Un ensemble de machines qui travaillent ensemble comme s'il s'agissait d'un seul système. Dans ce TP, **Minikube** simule ce cluster sur un seul ordinateur.

Nœud (Node) : Une machine physique ou virtuelle faisant partie du cluster et capable d'exécuter des tâches informatiques.

Minikube : Un outil spécifique qui permet de créer et de gérer un cluster Kubernetes local pour le développement et les tests.

Kubectl : L'outil de ligne de commande utilisé pour envoyer des instructions au cluster Kubernetes

Pod : La plus petite unité de mesure dans Kubernetes. C'est une "enveloppe" qui contient une **instance** de l'application (ici, un serveur web Nginx).

Instance : Un exemplaire unique d'un logiciel en cours d'exécution. Si vous lancez trois copies du même serveur, vous avez trois instances.

Déploiement (Deployment) : Un fichier ou une commande qui indique à Kubernetes comment l'application doit être installée

Réplica : Une copie identique d'un **Pod**. Le nombre de réplicas définit combien de fois l'application est dupliquée pour assurer la robustesse du système.

Mise à l'échelle (Scaling) : L'action d'augmenter ou de diminuer le nombre d'instances d'une application pour s'adapter au nombre d'utilisateurs.

Auto-guérison (Self-healing) : La capacité du système à détecter qu'une instance ne fonctionne plus et à la remplacer automatiquement par une nouvelle sans intervention humaine.

Équilibrage de charge (Load Balancing) : Un mécanisme qui répartit équitablement le trafic des utilisateurs entre les différentes instances disponibles pour éviter qu'une seule machine ne soit surchargée.

Tolérance aux fautes : La capacité d'un système à continuer de fonctionner malgré des erreurs ou des pannes matérielles.